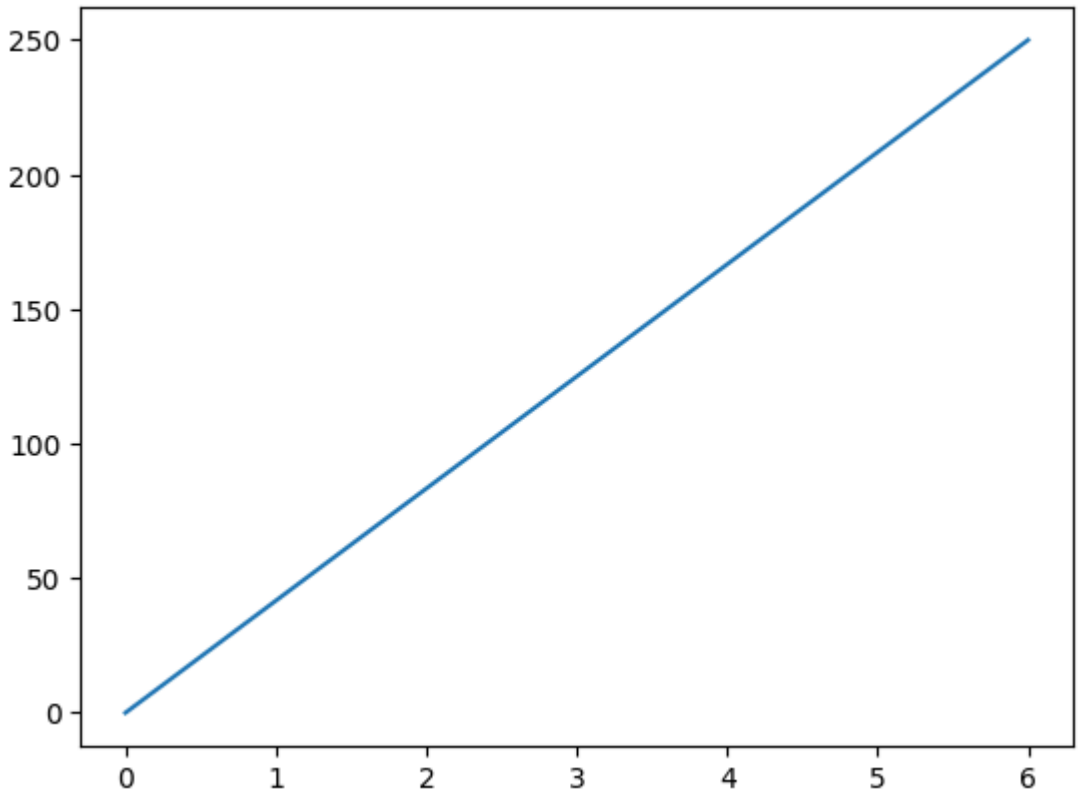
LINE PLOT

Plotting x and y points

In [1]:
```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([0, 6])
ypoints = np.array([0, 250])

plt.plot(xpoints, ypoints)
plt.show()
```



How Matplotlib Creates a Line from Two Points

The Math Behind the Line:

When you provide two points to `plt.plot()`, matplotlib automatically:

1. **Connects the points with a straight line**
2. **Calculates the slope** between the points
3. **Draws the line segment** connecting them

Your Example:

- Point 1: (0, 0)
- Point 2: (6, 250)

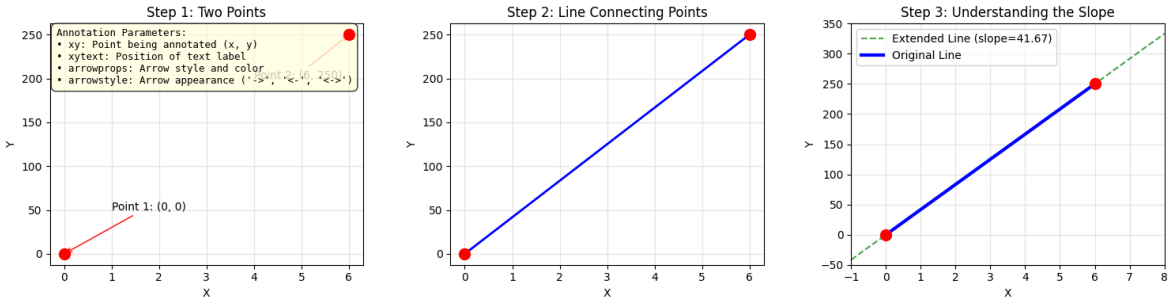
Slope Calculation:

```
Slope = (y₂ - y₁) / (x₂ - x₁)
Slope = (250 - 0) / (6 - 0) = 250/6 ≈ 41.67
```

Line Equation: y = 41.67x + 0

In [ ]:
```python
# Demonstrating how matplotlib connects two points

import matplotlib.pyplot as plt
import numpy as np

# Your original points
xpoints = np.array([0, 6])
ypoints = np.array([0, 250])

# Create a figure with subplots to show the process
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 4))

# Plot 1: Just the points
ax1.scatter(xpoints, ypoints, color='red', s=100, zorder=5)
ax1.set_title('Step 1: Two Points')
ax1.set_xlabel('X')
ax1.set_ylabel('Y')
ax1.grid(True, alpha=0.3)
# Add annotations with parameter explanations
ax1.annotate('Point 1: (0, 0)', xy=(0, 0), xytext=(1, 50),
            arrowprops=dict(arrowstyle='->', color='red'))
ax1.annotate('Point 2: (6, 250)', xy=(6, 250), xytext=(4, 200),
            arrowprops=dict(arrowstyle='->', color='red'))

# Add text box explaining annotation parameters
param_text = """Annotation Parameters:
• xy: Point being annotated (x, y)
• xytext: Position of text label
• arrowprops: Arrow style and color
• arrowstyle: Arrow appearance ('->', '<-', '<->')"""

ax1.text(0.02, 0.98, param_text, transform=ax1.transAxes,
         bbox=dict(boxstyle="round,pad=0.5", facecolor="lightyellow", alp
         fontsize=9, verticalalignment='top', fontfamily='monospace')

# Plot 2: Points with line
ax2.plot(xpoints, ypoints, 'b-', linewidth=2)
ax2.scatter(xpoints, ypoints, color='red', s=100, zorder=5)
ax2.set_title('Step 2: Line Connecting Points')
ax2.set_xlabel('X')
ax2.set_ylabel('Y')
ax2.grid(True, alpha=0.3)

# Plot 3: Extended line showing the slope
x_extended = np.linspace(-1, 8, 100)
slope = (ypoints[1] - ypoints[0]) / (xpoints[1] - xpoints[0])
y_extended = slope * x_extended + ypoints[0]  # y = mx + b, where b = 0

ax3.plot(x_extended, y_extended, 'g--', alpha=0.7, label=f'Extended Line
ax3.plot(xpoints, ypoints, 'b-', linewidth=3, label='Original Line')
ax3.scatter(xpoints, ypoints, color='red', s=100, zorder=5)
ax3.set_title('Step 3: Understanding the Slope')
ax3.set_xlabel('X')
ax3.set_ylabel('Y')
```

```
ax3.grid(True, alpha=0.3)
ax3.legend()
ax3.set_xlim(-1, 8)
ax3.set_ylim(-50, 350)

plt.tight_layout()
plt.show()

# Calculate and display the slope
print(f"Slope calculation:")
print(f"Point 1: ({xpoints[0]}, {ypoints[0]})")
print(f"Point 2: ({xpoints[1]}, {ypoints[1]})")
print(f"Slope = (y₂ - y₁) / (x₂ - x₁)")
print(f"Slope = ({ypoints[1]} - {ypoints[0]}) / ({xpoints[1]} - {xpoints[
print(f"Slope = {slope:.2f}")
print(f"Line equation: y = {slope:.2f}x + 0")
```



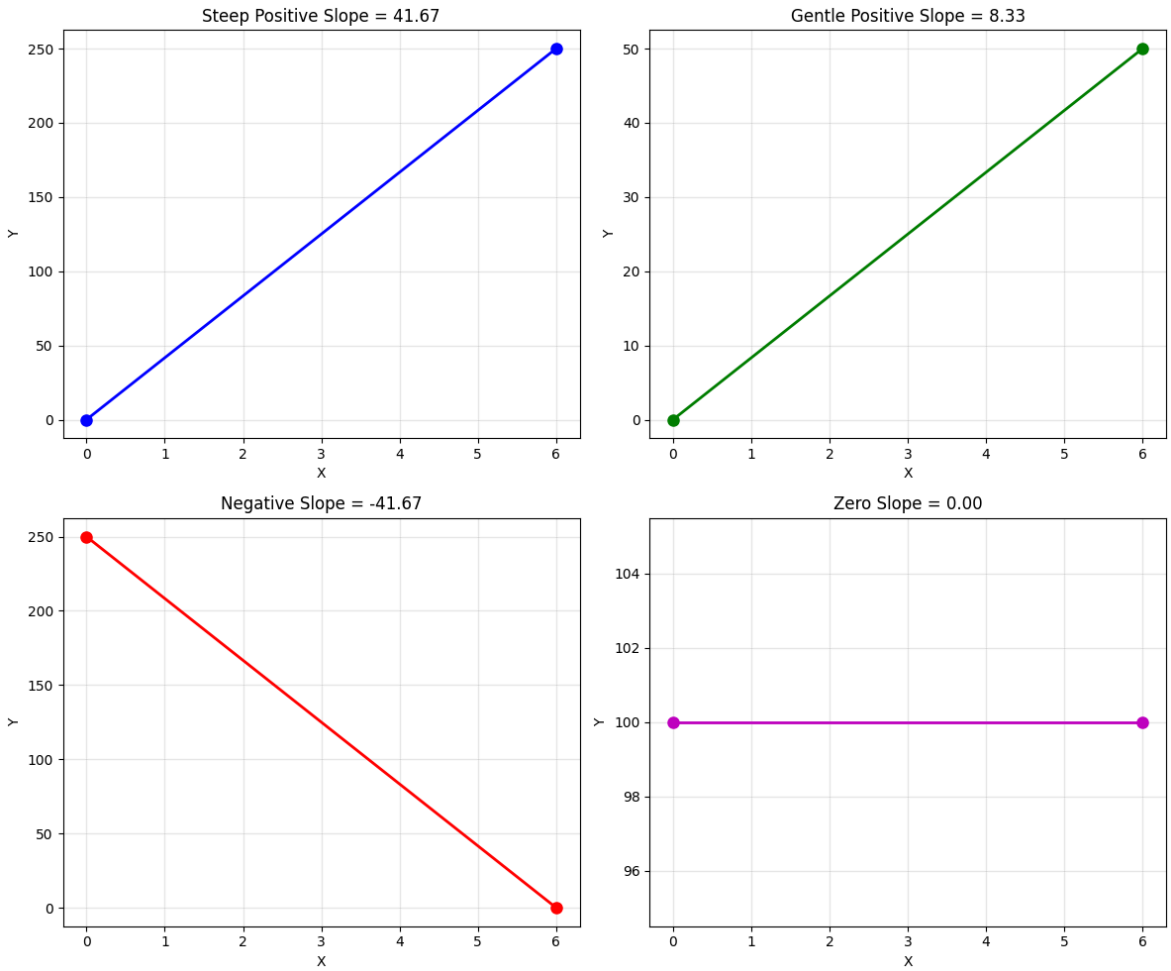

```
Slope calculation:
Point 1: (0, 0)
Point 2: (6, 250)
Slope = (y₂ - y₁) / (x₂ - x₁)
Slope = (250 - 0) / (6 - 0)
Slope = 41.67
Line equation: y = 41.67x + 0
```

In [23]:

```
# Comparing different slopes with various point combinations

fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(12, 10))

# Example 1: Your original points (steep positive slope)
x1, y1 = np.array([0, 6]), np.array([0, 250])
slope1 = (y1[1] - y1[0]) / (x1[1] - x1[0])
ax1.plot(x1, y1, 'b-', linewidth=2, marker='o', markersize=8)
ax1.set_title(f'Steep Positive Slope = {slope1:.2f}')
ax1.grid(True, alpha=0.3)
ax1.set_xlabel('X')
ax1.set_ylabel('Y')

# Example 2: Gentle positive slope
x2, y2 = np.array([0, 6]), np.array([0, 50])
slope2 = (y2[1] - y2[0]) / (x2[1] - x2[0])
ax2.plot(x2, y2, 'g-', linewidth=2, marker='o', markersize=8)
ax2.set_title(f'Gentle Positive Slope = {slope2:.2f}')
ax2.grid(True, alpha=0.3)
ax2.set_xlabel('X')
ax2.set_ylabel('Y')

# Example 3: Negative slope
x3, y3 = np.array([0, 6]), np.array([250, 0])
slope3 = (y3[1] - y3[0]) / (x3[1] - x3[0])
ax3.plot(x3, y3, 'r-', linewidth=2, marker='o', markersize=8)
```

```
ax3.set_title(f'Negative Slope = {slope3:.2f}')
ax3.grid(True, alpha=0.3)
ax3.set_xlabel('X')
ax3.set_ylabel('Y')

# Example 4: Zero slope (horizontal line)
x4, y4 = np.array([0, 6]), np.array([100, 100])
slope4 = (y4[1] - y4[0]) / (x4[1] - x4[0])
ax4.plot(x4, y4, 'm-', linewidth=2, marker='o', markersize=8)
ax4.set_title(f'Zero Slope = {slope4:.2f}')
ax4.grid(True, alpha=0.3)
ax4.set_xlabel('X')
ax4.set_ylabel('Y')

plt.tight_layout()
plt.show()

print("Slope Interpretation:")
print("• Positive slope: Line goes up from left to right")
print("• Negative slope: Line goes down from left to right")
print("• Zero slope: Horizontal line (no change in y)")
print("• Larger absolute value: Steeper line")
print("• Slope = rise/run = change in y / change in x")
```

```
[0 6]
[  0 250]
```



```
Slope Interpretation:
• Positive slope: Line goes up from left to right
• Negative slope: Line goes down from left to right
• Zero slope: Horizontal line (no change in y)
• Larger absolute value: Steeper line
• Slope = rise/run = change in y / change in x
```

What Happens with More Than Two Points?

When you provide **more than two points**, matplotlib:

- **Connects consecutive points** with straight line segments
- **Does NOT calculate a single slope** - each segment has its own slope
- **Creates a path** that follows the order of points

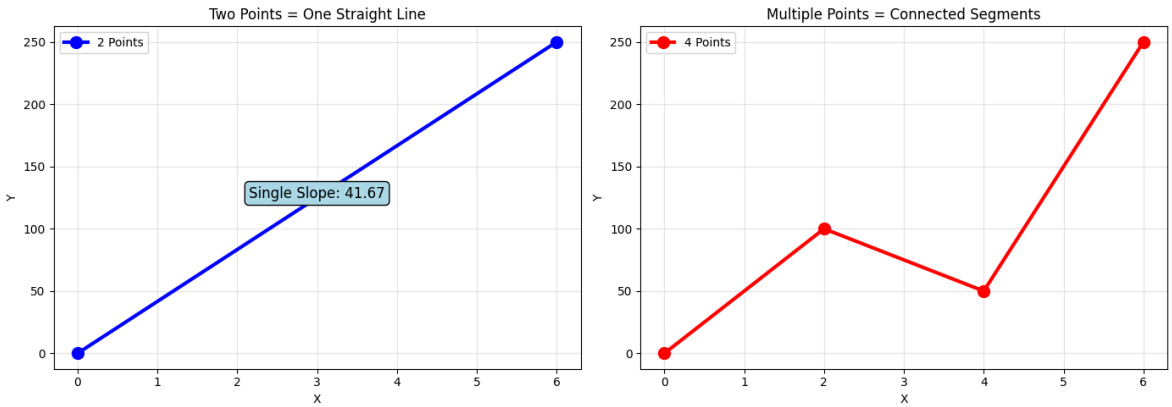
Key Differences:

- **2 points**: Creates one straight line with constant slope
- **3+ points**: Creates connected line segments (piecewise linear)

Let's see this in action:

In [ ]:
```
# Demonstration: Two points vs Multiple points

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Two points - single straight line
x_two = np.array([0, 6])
y_two = np.array([0, 250])

ax1.plot(x_two, y_two, 'b-', linewidth=3, marker='o', markersize=10, labe
ax1.set_title('Two Points = One Straight Line')
ax1.set_xlabel('X')
ax1.set_ylabel('Y')
ax1.grid(True, alpha=0.3)
ax1.legend()

# Annotate slope
slope_two = (y_two[1] - y_two[0]) / (x_two[1] - x_two[0])
ax1.text(3, 125, f'Single Slope: {slope_two:.2f}',
         bbox=dict(boxstyle="round,pad=0.3", facecolor="lightblue"),
         fontsize=12, ha='center')

# Multiple points - connected line segments
x_multi = np.array([0, 2, 4, 6])
y_multi = np.array([0, 100, 50, 250])

ax2.plot(x_multi, y_multi, 'r-', linewidth=3, marker='o', markersize=10,
ax2.set_title('Multiple Points = Connected Segments')
ax2.set_xlabel('X')
ax2.set_ylabel('Y')
ax2.grid(True, alpha=0.3)
ax2.legend()

# Annotate different slopes for each segment
slopes = []
for i in range(len(x_multi)-1):
```

```
    slope = (y_multi[i+1] - y_multi[i]) / (x_multi[i+1] - x_multi[i])
    slopes.append(slope)
    mid_x = (x_multi[i] + x_multi[i+1]) / 2
    mid_y = (y_multi[i] + y_multi[i+1]) / 2
    ax2.text(mid_x, mid_y + 20, f'Slope: {slope:.1f}',
             bbox=dict(boxstyle="round,pad=0.2", facecolor="lightcoral",
             fontsize=10, ha='center')

plt.tight_layout()
plt.show()

# Print slope calculations
print("Two Points Analysis:")
print(f"Point 1: ({x_two[0]}, {y_two[0]}), Point 2: ({x_two[1]}, {y_two[1
print(f"Single slope: {slope_two:.2f}")
print()

print("Multiple Points Analysis:")
for i in range(len(x_multi)-1):
    print(f"Segment {i+1}: Point ({x_multi[i]}, {y_multi[i]}) to ({x_mult
    print(f"Slope: {slopes[i]:.2f}")
print()

print("Key Insight:")
print("• Two points always create ONE straight line with constant slope")
print("• Multiple points create MULTIPLE segments with different slopes")
print("• Matplotlib connects points in the order you provide them")
```

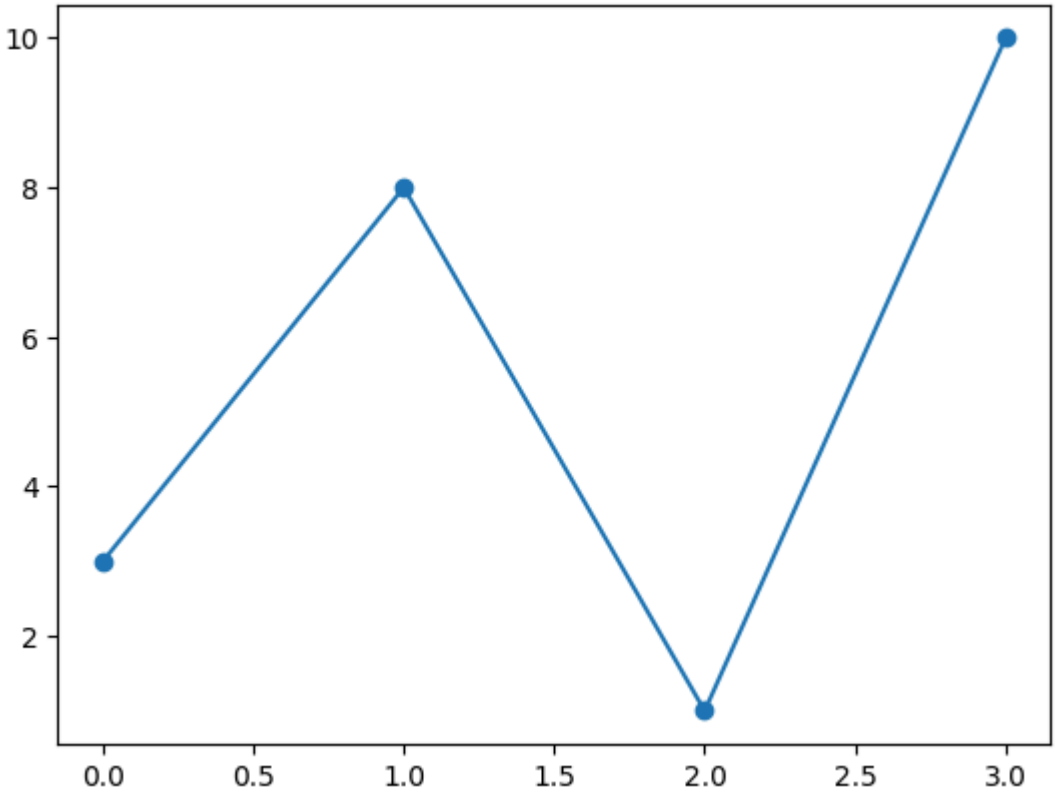



```
Two Points Analysis:
Point 1: (0, 0), Point 2: (6, 250)
Single slope: 41.67

Multiple Points Analysis:
Segment 1: Point (0, 0) to (2, 100)
Segment 2: Point (2, 100) to (4, 50)
Segment 3: Point (4, 50) to (6, 250)

Key Insight:
• Two points always create ONE straight line with constant slope
• Multiple points create MULTIPLE segments with different slopes
• Matplotlib connects points in the order you provide them
```

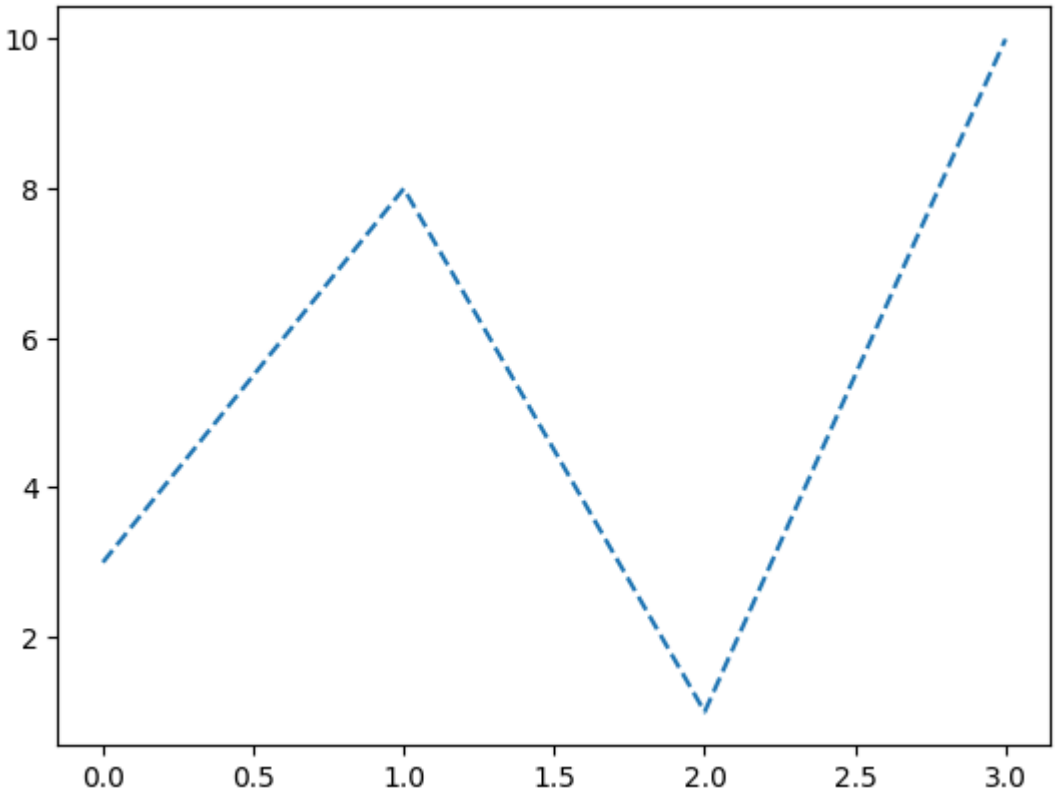
Markers

Linestyle

In [5]:
```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, marker = 'o')
plt.show()
```

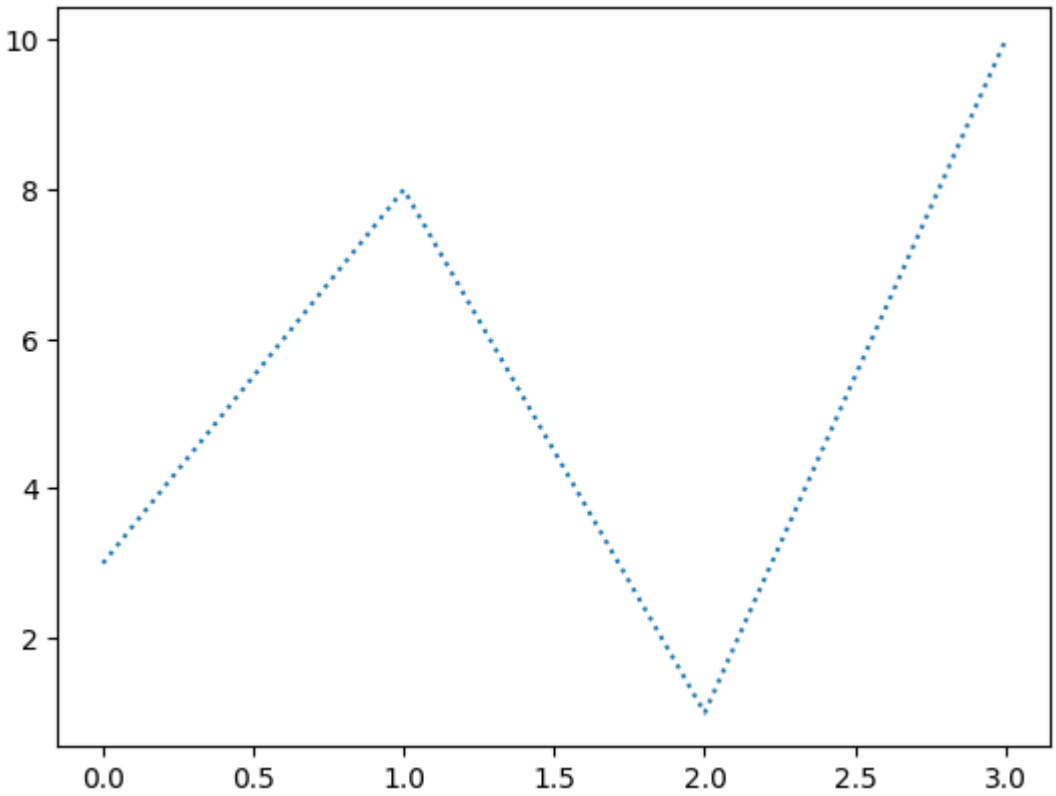


In [6]:
```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, linestyle = 'dashed')
plt.show()
```

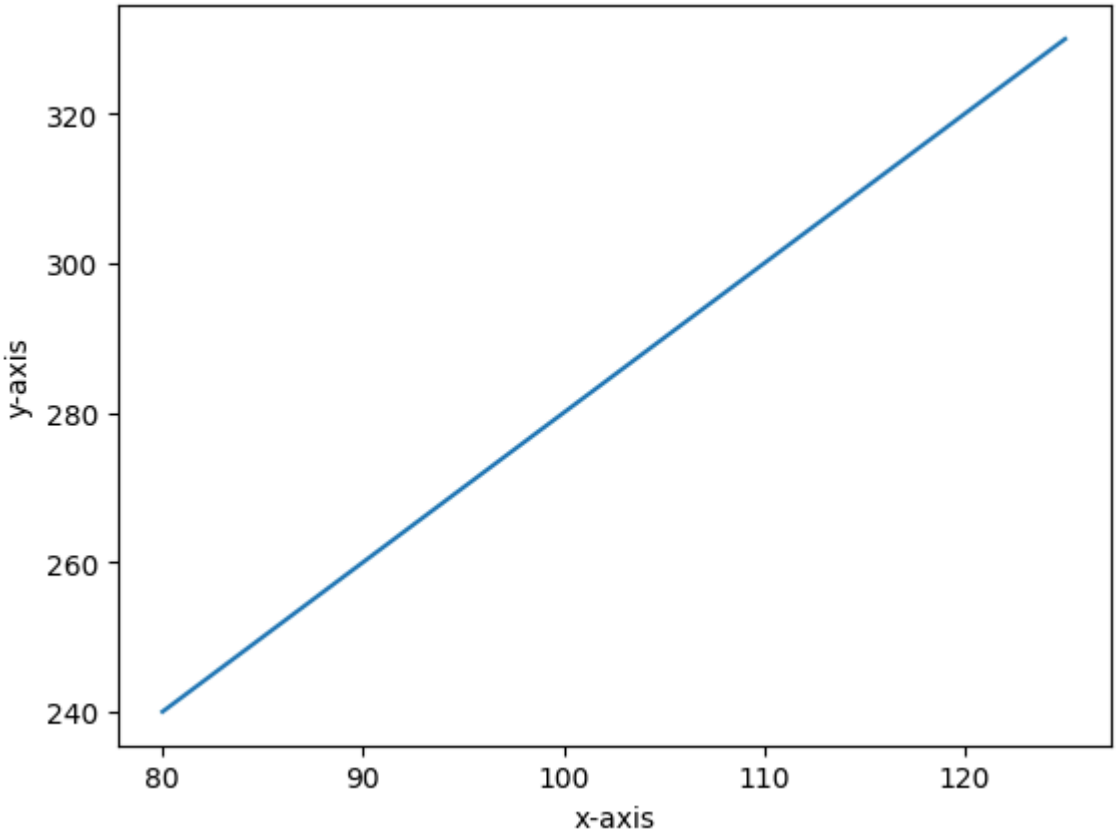


Create Labels for a Plot

In [7]:
```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, linestyle = 'dotted')
plt.show()
```

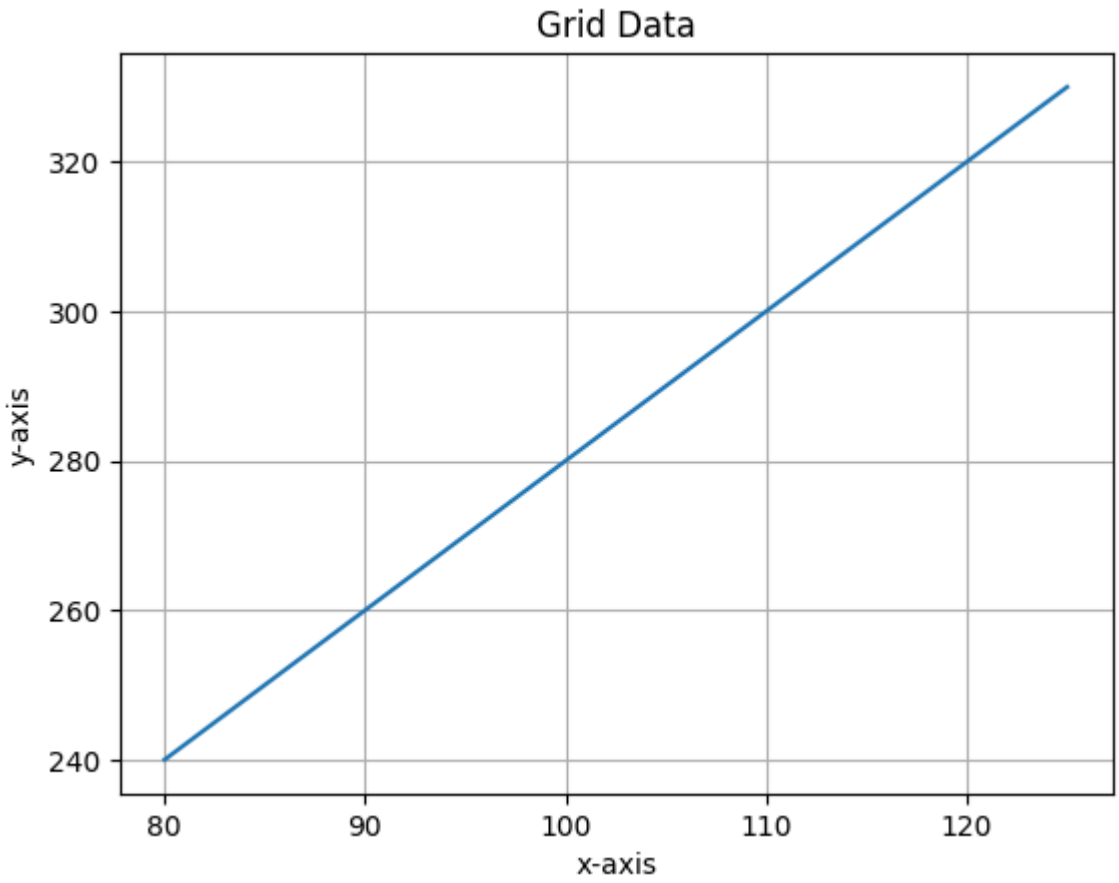


In [8]:
```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.plot(x, y)

plt.xlabel("x-axis")
plt.ylabel("y-axis")

plt.show()
```



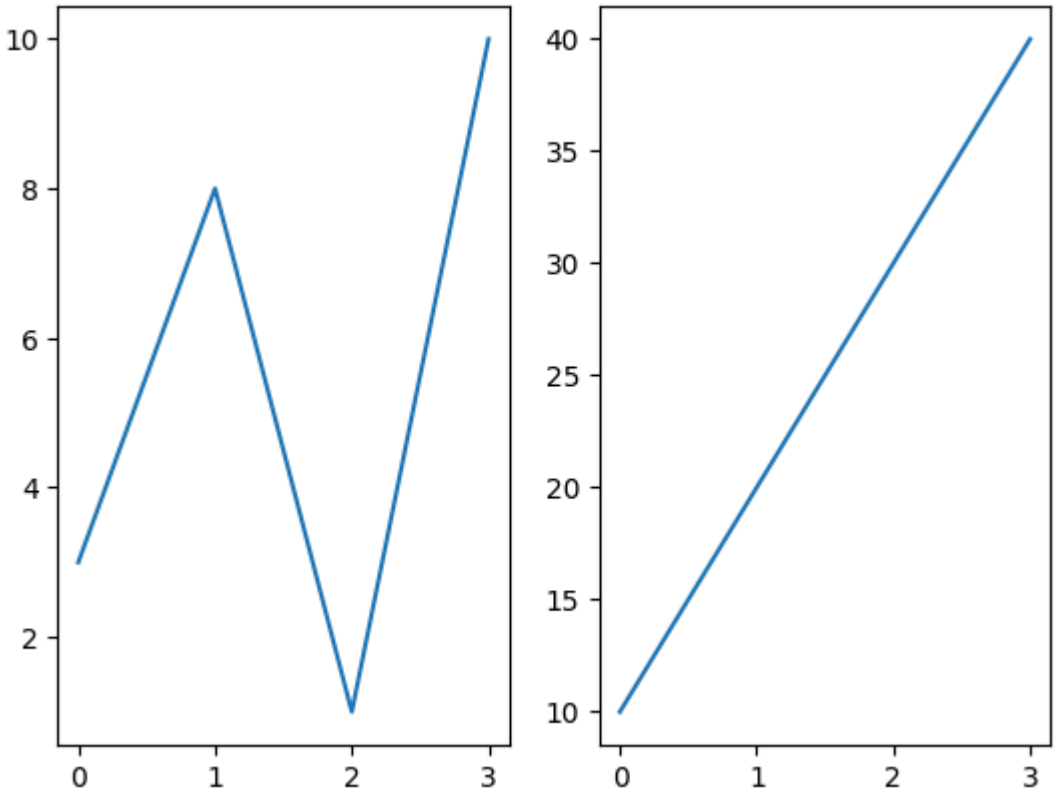
Grid Lines to a Plot

In [9]:
```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.title("Grid Data")
plt.xlabel("x-axis")
plt.ylabel("y-axis")

plt.plot(x, y)

plt.grid()

plt.show()
```



Display Multiple Plots

In [10]:
```
import matplotlib.pyplot as plt
import numpy as np

#plot 1:
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(1, 2, 1)
plt.plot(x,y)

#plot 2:
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(1, 2, 2)
plt.plot(x,y)

plt.show()
```



Understanding Matplotlib Subplots

What are Subplots? Subplots allow you to display **multiple plots in a single figure**, arranged in a grid layout.

Your Code Analysis:

```
plt.subplot(1, 2, 1)  # Create subplot at position 1
plt.subplot(1, 2, 2)  # Create subplot at position 2
```

Subplot Parameters:

- `plt.subplot(nrows, ncols, index)`
- **nrows**: Number of rows in the grid
- **ncols**: Number of columns in the grid
- **index**: Position of current subplot (starts from 1)

Your Example:

- `(1, 2, 1)` : 1 row, 2 columns, position 1 (left plot)
- `(1, 2, 2)` : 1 row, 2 columns, position 2 (right plot)

Grid Layout:

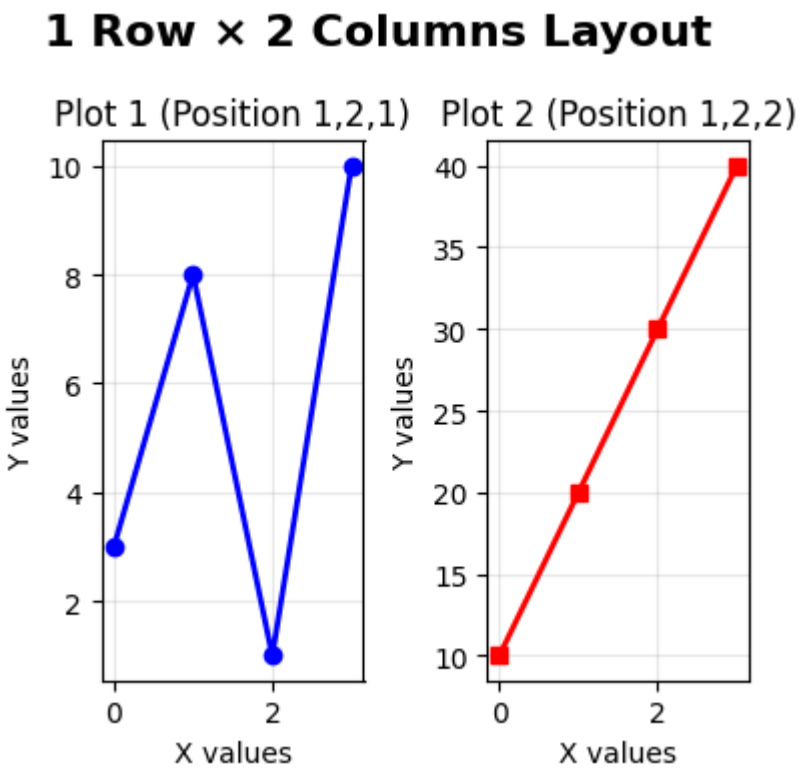
```
┌─────────┬─────────┐
│ Plot 1  │ Plot 2  │
│ (1,2,1) │ (1,2,2) │
└─────────┴─────────┘
```

In [ ]:
```
# Demonstrating Different Subplot Configurations
```

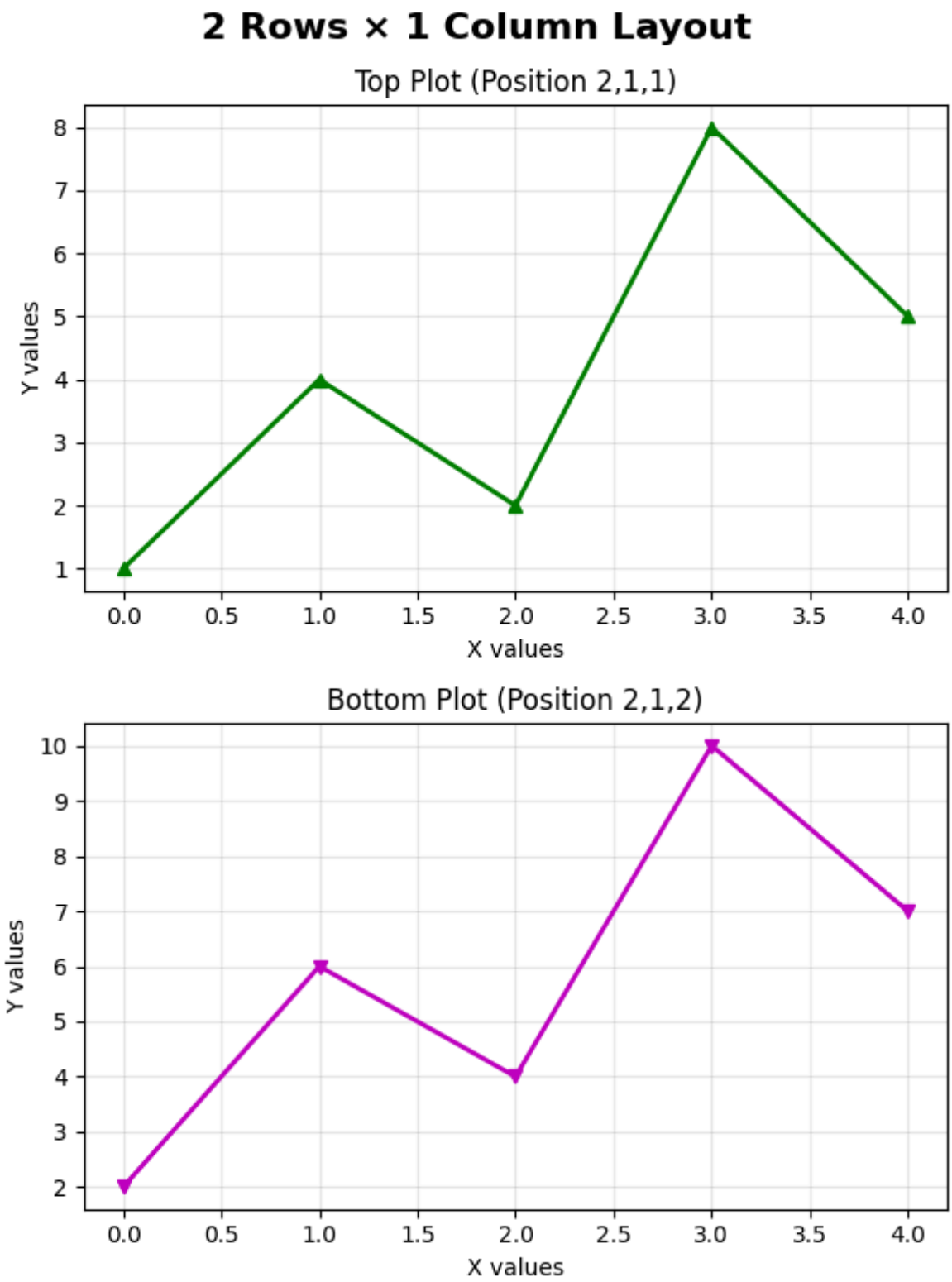
```python
import matplotlib.pyplot as plt
import numpy as np

# Sample data for demonstrations
x = np.array([0, 1, 2, 3, 4])
y1 = np.array([1, 4, 2, 8, 5])
y2 = np.array([2, 6, 4, 10, 7])
y3 = np.array([3, 5, 7, 6, 9])
y4 = np.array([4, 8, 6, 12, 10])

print("=== CONFIGURATION 1: Your Original Example (1 row, 2 columns) ===")
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)  # Row 1, Column 1
plt.plot([0, 1, 2, 3], [3, 8, 1, 10], 'b-o', linewidth=2)
plt.title('Plot 1 (Position 1,2,1)')
plt.xlabel('X values')
plt.ylabel('Y values')
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)  # Row 1, Column 2
plt.plot([0, 1, 2, 3], [10, 20, 30, 40], 'r-s', linewidth=2)
plt.title('Plot 2 (Position 1,2,2)')
plt.xlabel('X values')
plt.ylabel('Y values')
plt.grid(True, alpha=0.3)

plt.suptitle('1 Row × 2 Columns Layout', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()

print("\n=== CONFIGURATION 2: 2 rows, 1 column (vertical stack) ===")
plt.figure(figsize=(6, 8))

plt.subplot(2, 1, 1)  # Row 1, Column 1
plt.plot(x, y1, 'g-^', linewidth=2)
plt.title('Top Plot (Position 2,1,1)')
plt.xlabel('X values')
plt.ylabel('Y values')
plt.grid(True, alpha=0.3)

plt.subplot(2, 1, 2)  # Row 2, Column 1
plt.plot(x, y2, 'm-v', linewidth=2)
plt.title('Bottom Plot (Position 2,1,2)')
plt.xlabel('X values')
plt.ylabel('Y values')
plt.grid(True, alpha=0.3)

plt.suptitle('2 Rows × 1 Column Layout', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()

print("\n=== CONFIGURATION 3: 2 rows, 2 columns (2×2 grid) ===")
plt.figure(figsize=(10, 8))

plt.subplot(2, 2, 1)  # Top-left
plt.plot(x, y1, 'b-o')
plt.title('Top-Left (2,2,1)')
plt.grid(True, alpha=0.3)
```

```
plt.subplot(2, 2, 2)  # Top-right
plt.plot(x, y2, 'r-s')
plt.title('Top-Right (2,2,2)')
plt.grid(True, alpha=0.3)

plt.subplot(2, 2, 3)  # Bottom-left
plt.plot(x, y3, 'g-^')
plt.title('Bottom-Left (2,2,3)')
plt.grid(True, alpha=0.3)

plt.subplot(2, 2, 4)  # Bottom-right
plt.plot(x, y4, 'm-v')
plt.title('Bottom-Right (2,2,4)')
plt.grid(True, alpha=0.3)

plt.suptitle('2 Rows × 2 Columns Layout', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()
```

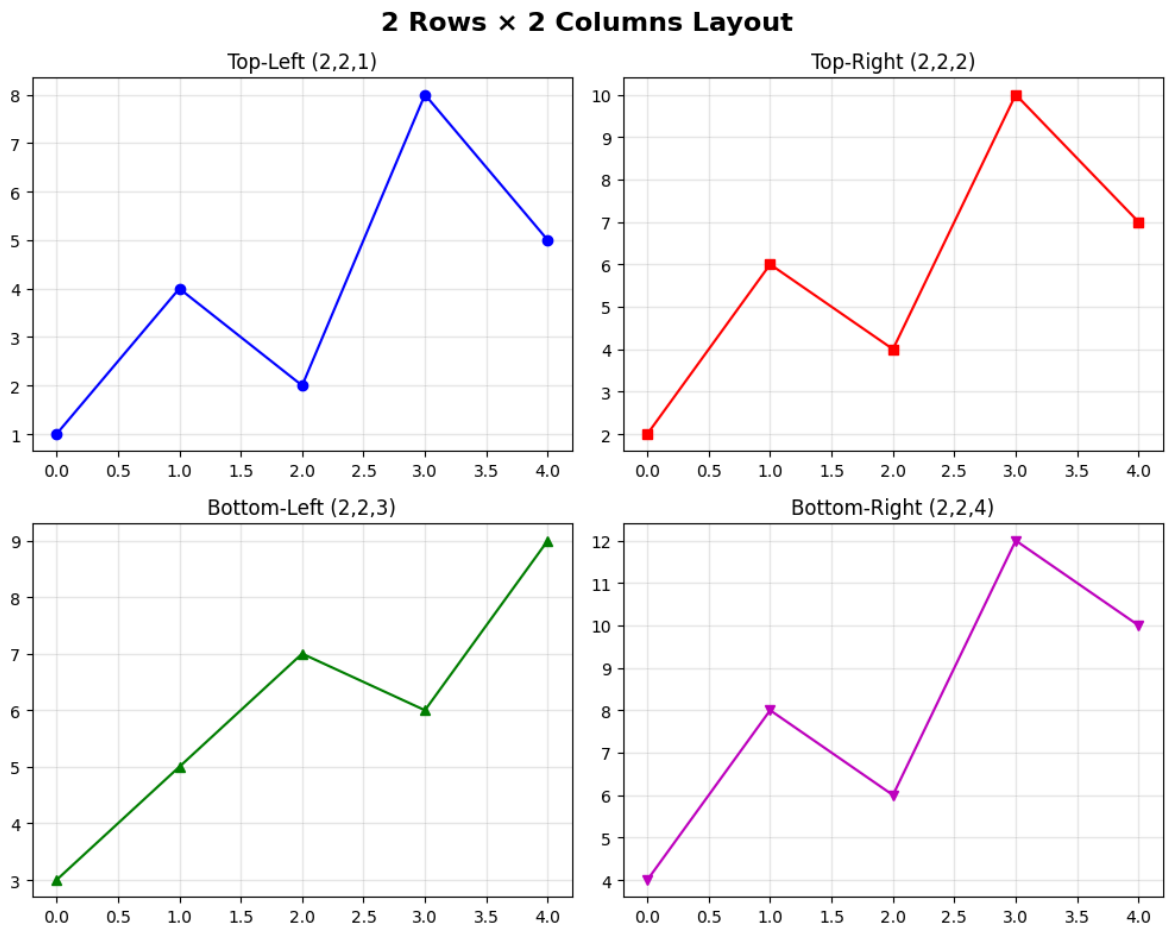
```
=== CONFIGURATION 1: Your Original Example (1 row, 2 columns) ===
```




```
=== CONFIGURATION 2: 2 rows, 1 column (vertical stack) ===
```



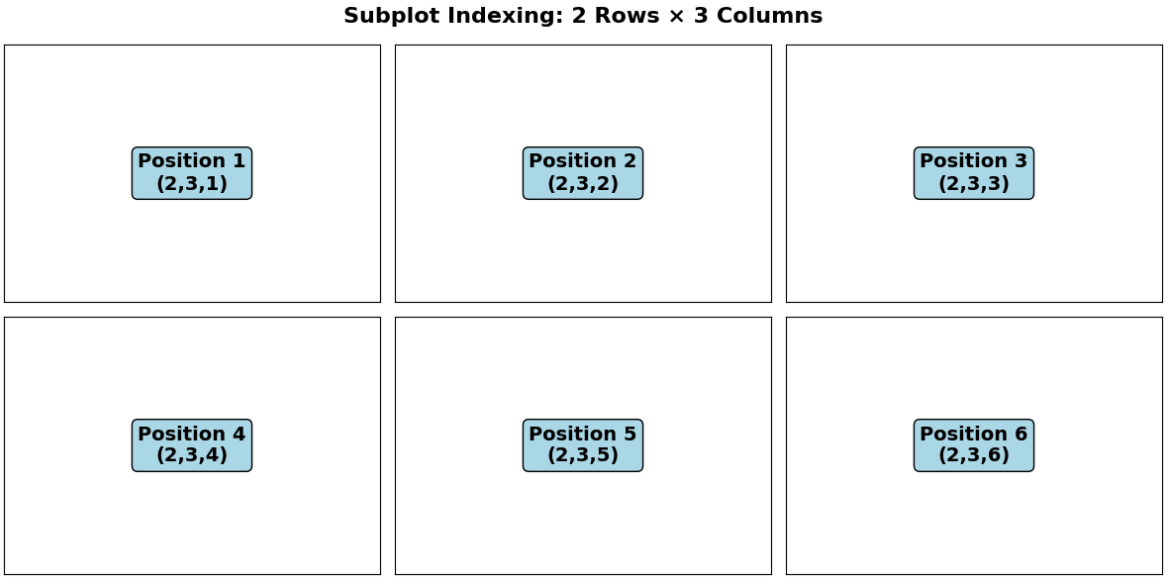

```
=== CONFIGURATION 3: 2 rows, 2 columns (2×2 grid) ===
```



In [12]:
```
# Visual Guide to Subplot Indexing

print("=== SUBPLOT INDEXING SYSTEM ===")
print("\nFor a 2×3 grid (2 rows, 3 columns):")
print("┌───┬───┬───┐")
print("│ 1 │ 2 │ 3 │  ← Row 1")
print("├───┼───┼───┤")
print("│ 4 │ 5 │ 6 │  ← Row 2")
print("└───┴───┴───┘")
print("  ↑   ↑   ↑")
print("Col1 Col2 Col3")

print("\nIndex Formula: index = (row-1) × ncols + col")
print("Example: Position (2,2) in 2×3 grid = (2-1)×3 + 2 = 5")

# Demonstrate with actual subplot
fig = plt.figure(figsize=(12, 6))

for i in range(1, 7):
    plt.subplot(2, 3, i)
    plt.text(0.5, 0.5, f'Position {i}\n(2,3,{i})',
             ha='center', va='center', fontsize=14, fontweight='bold',
             bbox=dict(boxstyle="round,pad=0.3", facecolor="lightblue"))
    plt.xlim(0, 1)
    plt.ylim(0, 1)
    plt.xticks([])
    plt.yticks([])

plt.suptitle('Subplot Indexing: 2 Rows × 3 Columns', fontsize=16, fontwei
plt.tight_layout()
plt.show()
```

```
print("\n=== KEY SUBPLOT RULES ===")
print("1. Indexing starts from 1 (not 0)")
print("2. Goes left-to-right, then top-to-bottom")
print("3. plt.subplot(nrows, ncols, index)")
print("4. Can also write as plt.subplot(nci) where n=rows, c=cols, i=inde
print("   Example: plt.subplot(231) same as plt.subplot(2,3,1)")
```

```
=== SUBPLOT INDEXING SYSTEM ===

For a 2×3 grid (2 rows, 3 columns):

┌───┬───┬───┐
│ 1 │ 2 │ 3 │  ← Row 1
├───┼───┼───┤
│ 4 │ 5 │ 6 │  ← Row 2
└───┴───┴───┘
  ↑   ↑   ↑
Col1 Col2 Col3

Index Formula: index = (row-1) × ncols + col
Example: Position (2,2) in 2×3 grid = (2-1)×3 + 2 = 5
```




```
=== KEY SUBPLOT RULES ===
1. Indexing starts from 1 (not 0)
2. Goes left-to-right, then top-to-bottom
3. plt.subplot(nrows, ncols, index)
4. Can also write as plt.subplot(nci) where n=rows, c=cols, i=index
   Example: plt.subplot(231) same as plt.subplot(2,3,1)
```

Alternative Subplot Methods

Method 1: plt.subplots() (Recommended Modern Approach)

Instead of calling `plt.subplot()` multiple times, you can create all subplots at once:

```
fig, axes = plt.subplots(nrows, ncols, figsize=(width, height))
```

Advantages:

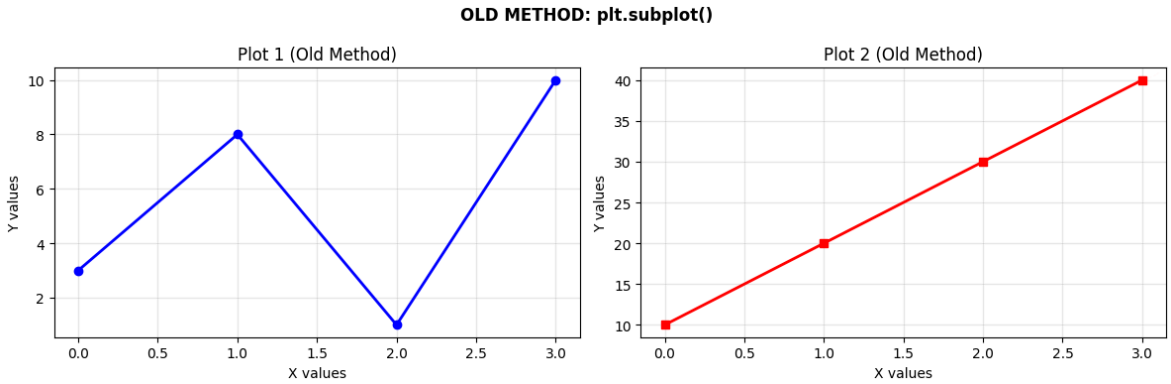
- Creates all subplots at once
- Returns figure and axes objects for easier manipulation

- Better control over spacing and layout
- More Pythonic and cleaner code

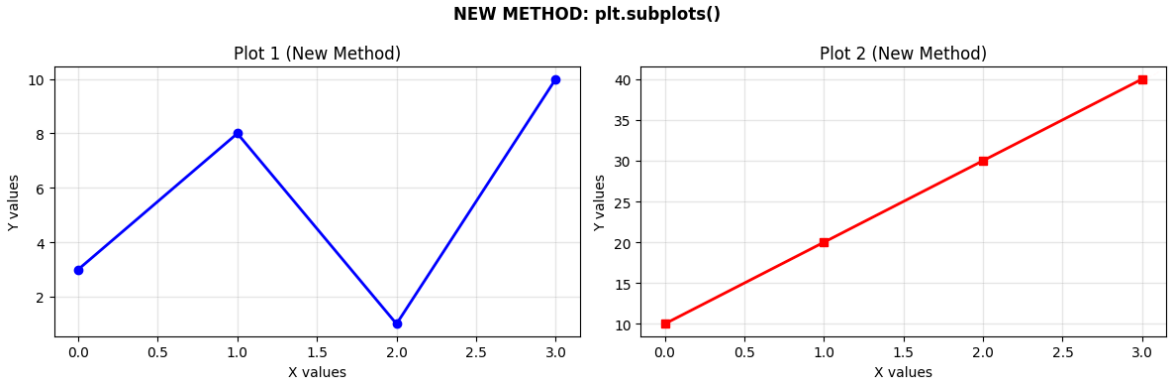
In [13]:
```
# Comparing Old vs New Subplot Methods

# Sample data
x = np.array([0, 1, 2, 3])
y1 = np.array([3, 8, 1, 10])
y2 = np.array([10, 20, 30, 40])

print("=== OLD METHOD: Using plt.subplot() ===")
plt.figure(figsize=(12, 4))

# Old way - your original code
plt.subplot(1, 2, 1)
plt.plot(x, y1, 'b-o', linewidth=2)
plt.title('Plot 1 (Old Method)')
plt.xlabel('X values')
plt.ylabel('Y values')
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(x, y2, 'r-s', linewidth=2)
plt.title('Plot 2 (Old Method)')
plt.xlabel('X values')
plt.ylabel('Y values')
plt.grid(True, alpha=0.3)

plt.suptitle('OLD METHOD: plt.subplot()', fontweight='bold')
plt.tight_layout()
plt.show()

print("\n=== NEW METHOD: Using plt.subplots() ===")
# New way - modern approach
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

# Plot on specific axes
ax1.plot(x, y1, 'b-o', linewidth=2)
ax1.set_title('Plot 1 (New Method)')
ax1.set_xlabel('X values')
ax1.set_ylabel('Y values')
ax1.grid(True, alpha=0.3)

ax2.plot(x, y2, 'r-s', linewidth=2)
ax2.set_title('Plot 2 (New Method)')
ax2.set_xlabel('X values')
ax2.set_ylabel('Y values')
ax2.grid(True, alpha=0.3)

fig.suptitle('NEW METHOD: plt.subplots()', fontweight='bold')
plt.tight_layout()
plt.show()

print("\n=== 2×2 GRID COMPARISON ===")
# More complex example with 2x2 grid
fig, axes = plt.subplots(2, 2, figsize=(10, 8))

# Sample data for 4 plots
data = [
```

```
    ([1, 2, 3, 4], [1, 4, 2, 3], 'Linear'),
    ([1, 2, 3, 4], [1, 4, 9, 16], 'Quadratic'),
    ([1, 2, 3, 4], [1, 1, 2, 6], 'Factorial-like'),
    ([1, 2, 3, 4], [2, 4, 8, 16], 'Exponential')
]

# Plot on each subplot using axes array
for i, (x_data, y_data, label) in enumerate(data):
    row, col = i // 2, i % 2  # Convert index to row, col
    axes[row, col].plot(x_data, y_data, 'o-', linewidth=2)
    axes[row, col].set_title(f'{label} (Position {i+1})')
    axes[row, col].grid(True, alpha=0.3)

fig.suptitle('2×2 Grid Using plt.subplots()', fontweight='bold')
plt.tight_layout()
plt.show()

print("\n=== CODE COMPARISON ===")
print("OLD WAY (plt.subplot):")
print("plt.subplot(1, 2, 1)")
print("plt.plot(x, y1)")
print("plt.subplot(1, 2, 2)")
print("plt.plot(x, y2)")
print()
print("NEW WAY (plt.subplots):")
print("fig, (ax1, ax2) = plt.subplots(1, 2)")
print("ax1.plot(x, y1)")
print("ax2.plot(x, y2)")
```

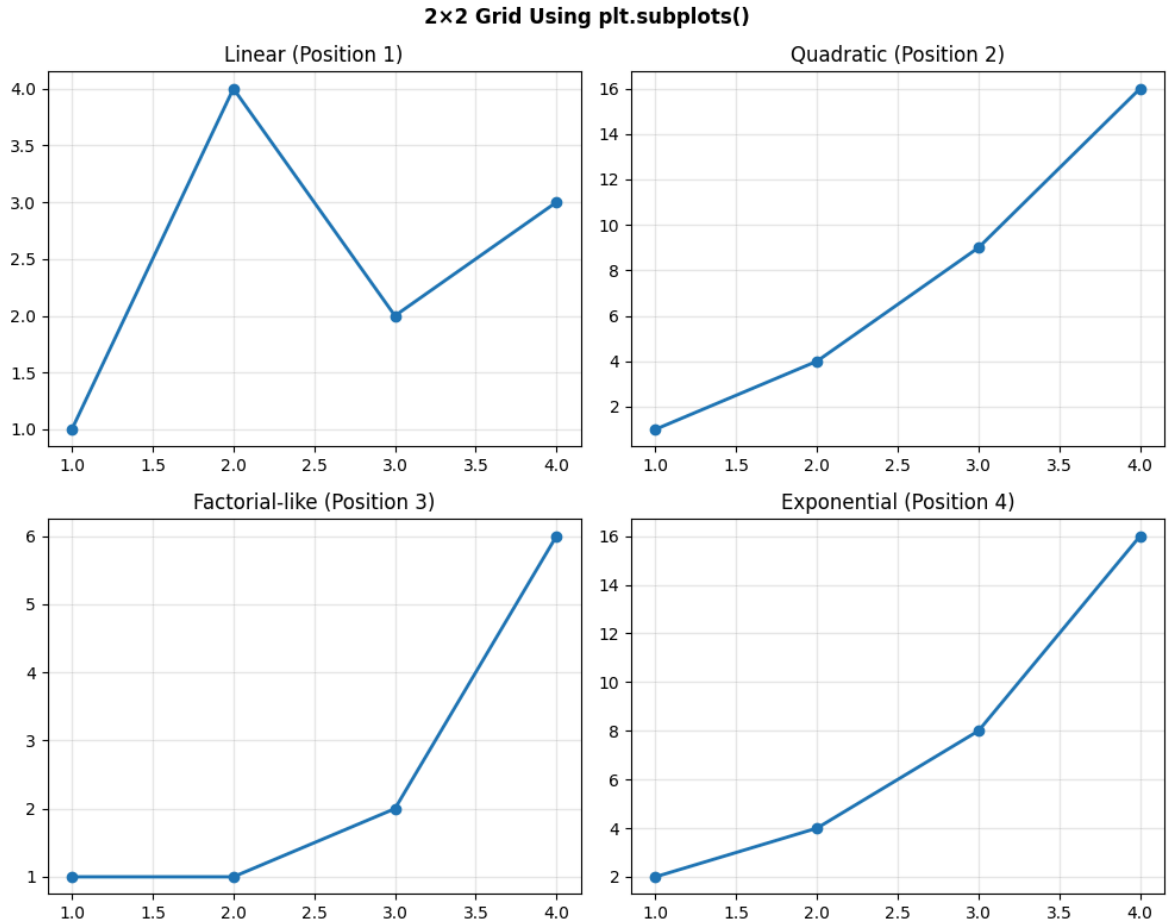
```
=== OLD METHOD: Using plt.subplot() ===
```




```
=== NEW METHOD: Using plt.subplots() ===
```




```
=== 2×2 GRID COMPARISON ===
```



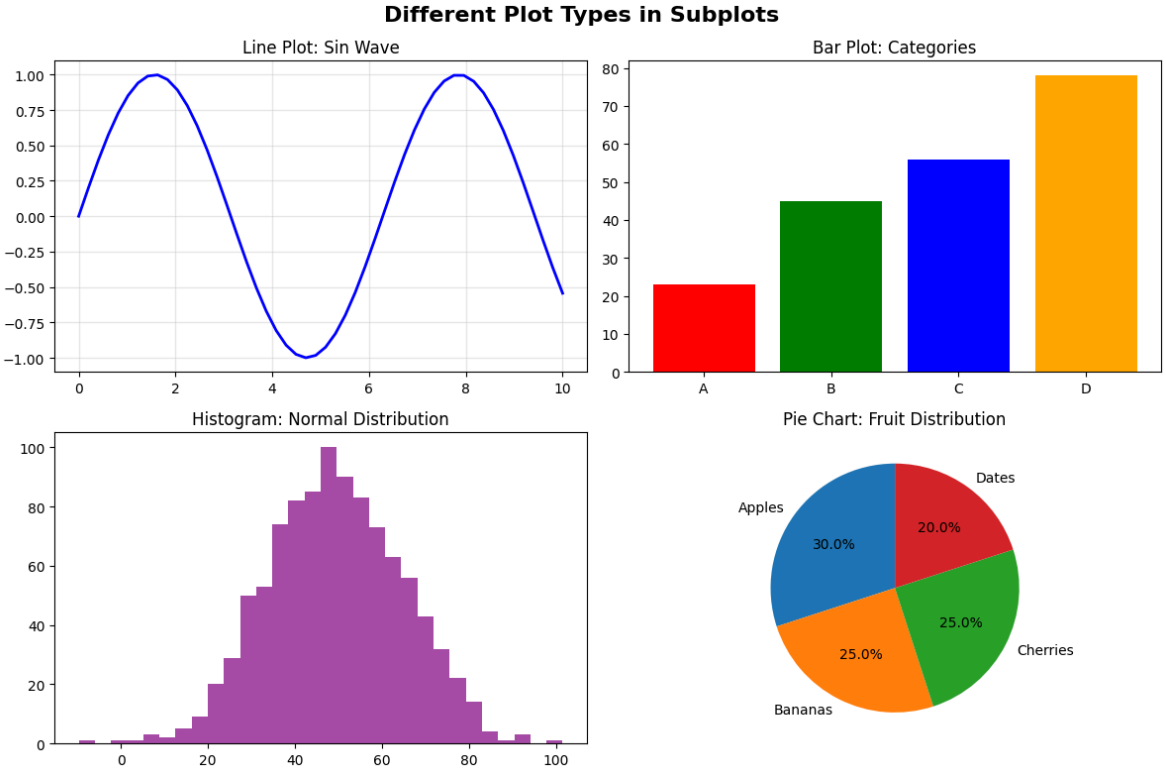

```
=== CODE COMPARISON ===
OLD WAY (plt.subplot):
plt.subplot(1, 2, 1)
plt.plot(x, y1)
plt.subplot(1, 2, 2)
plt.plot(x, y2)

NEW WAY (plt.subplots):
fig, (ax1, ax2) = plt.subplots(1, 2)
ax1.plot(x, y1)
ax2.plot(x, y2)
```

In [14]:

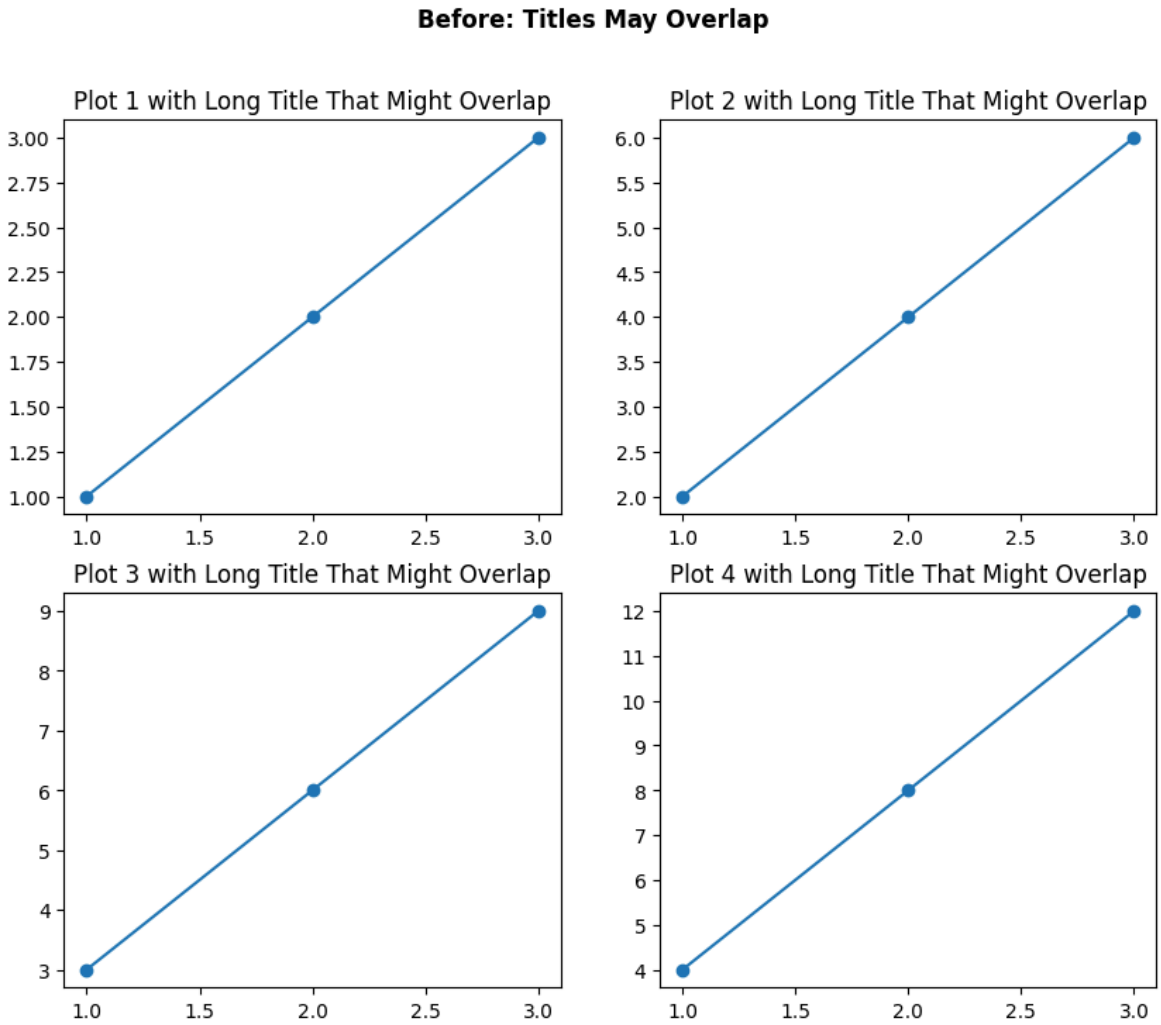
```python
# Practical Subplot Tips and Use Cases

print("=== PRACTICAL SUBPLOT TIPS ===")

# Tip 1: Different plot types in subplots
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

# Data for different visualizations
x = np.linspace(0, 10, 50)
y = np.sin(x)
categories = ['A', 'B', 'C', 'D']
values = [23, 45, 56, 78]
data_hist = np.random.normal(50, 15, 1000)
sizes = [30, 25, 25, 20]
labels = ['Apples', 'Bananas', 'Cherries', 'Dates']

# Line plot
axes[0, 0].plot(x, y, 'b-', linewidth=2)
axes[0, 0].set_title('Line Plot: Sin Wave')
axes[0, 0].grid(True, alpha=0.3)
```

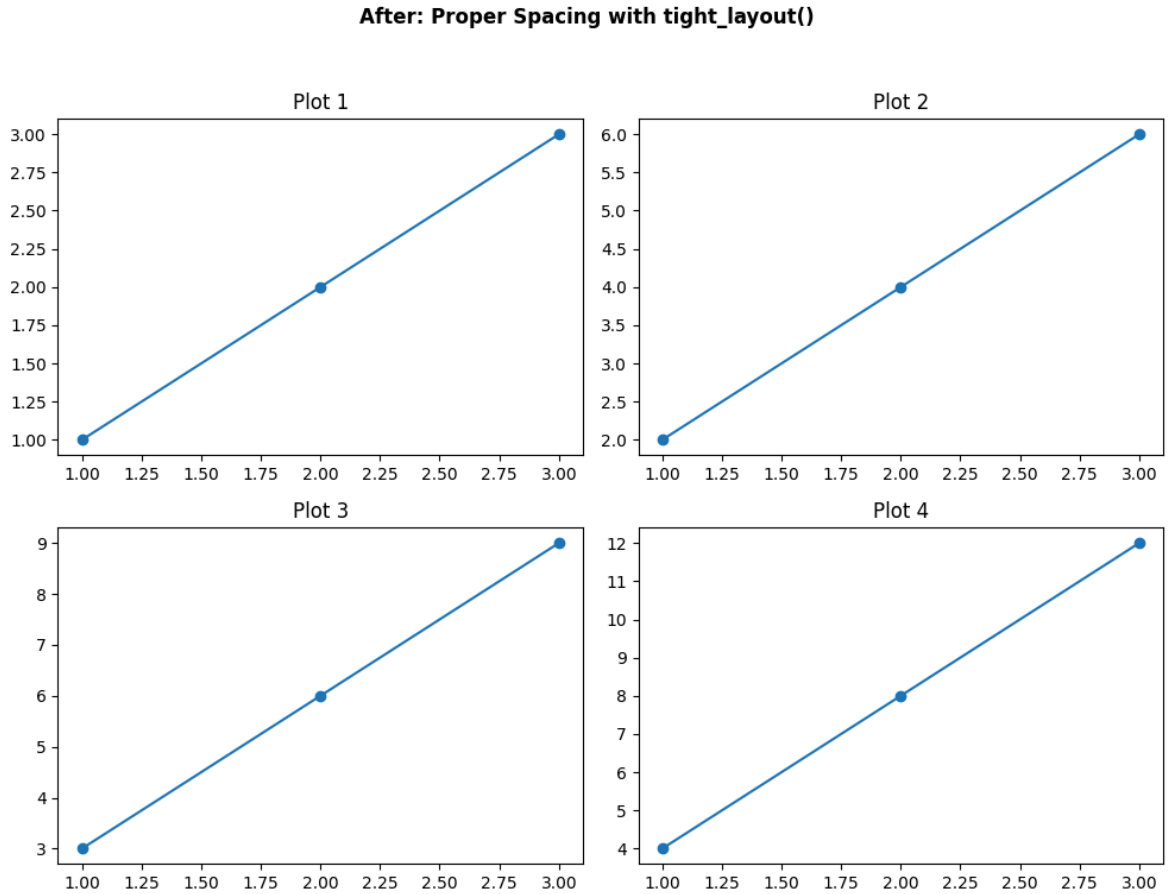
```
# Bar plot
axes[0, 1].bar(categories, values, color=['red', 'green', 'blue', 'orange
axes[0, 1].set_title('Bar Plot: Categories')

# Histogram
axes[1, 0].hist(data_hist, bins=30, alpha=0.7, color='purple')
axes[1, 0].set_title('Histogram: Normal Distribution')

# Pie chart
axes[1, 1].pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90)
axes[1, 1].set_title('Pie Chart: Fruit Distribution')

fig.suptitle('Different Plot Types in Subplots', fontsize=16, fontweight=
plt.tight_layout()
plt.show()

print("\n=== SPACING AND LAYOUT CONTROL ===")

# Demonstrate spacing control
fig, axes = plt.subplots(2, 2, figsize=(10, 8))

for i in range(4):
    row, col = i // 2, i % 2
    axes[row, col].plot([1, 2, 3], [i+1, (i+1)*2, (i+1)*3], 'o-')
    axes[row, col].set_title(f'Plot {i+1} with Long Title That Might Over

# Before adjustment
fig.suptitle('Before: Titles May Overlap', fontweight='bold')
plt.show()

# After adjustment with proper spacing
fig, axes = plt.subplots(2, 2, figsize=(10, 8))

for i in range(4):
    row, col = i // 2, i % 2
    axes[row, col].plot([1, 2, 3], [i+1, (i+1)*2, (i+1)*3], 'o-')
    axes[row, col].set_title(f'Plot {i+1}')

# Adjust spacing
fig.suptitle('After: Proper Spacing with tight_layout()', fontweight='bol
plt.tight_layout(rect=[0, 0.03, 1, 0.95])  # Leave space for suptitle
plt.show()

print("\n=== WHEN TO USE SUBPLOTS ===")
print("✅ Comparing related datasets")
print("✅ Showing different views of same data")
print("✅ Before/after comparisons")
print("✅ Different plot types for comprehensive analysis")
print("✅ Time series with different time periods")
print("✅ A/B testing results")
print("✅ Multi-variable analysis")

print("\n=== SUBPLOT BEST PRACTICES ===")
print("• Use consistent scales when comparing data")
print("• Add clear titles to each subplot")
print("• Use plt.tight_layout() to prevent overlap")
print("• Consider color consistency across subplots")
print("• Don't overcrowd - limit to 2×2 or 2×3 for readability")
print("• Use fig.suptitle() for overall figure title")
```

```
=== PRACTICAL SUBPLOT TIPS ===
```




```
=== SPACING AND LAYOUT CONTROL ===
```

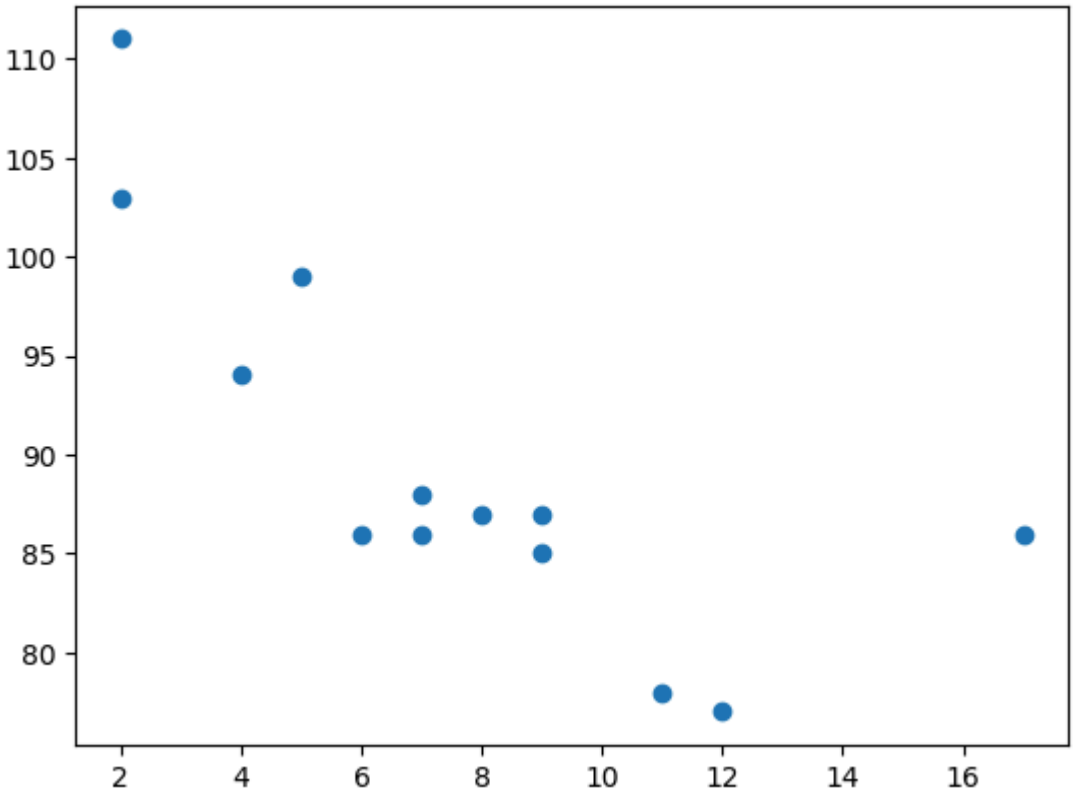




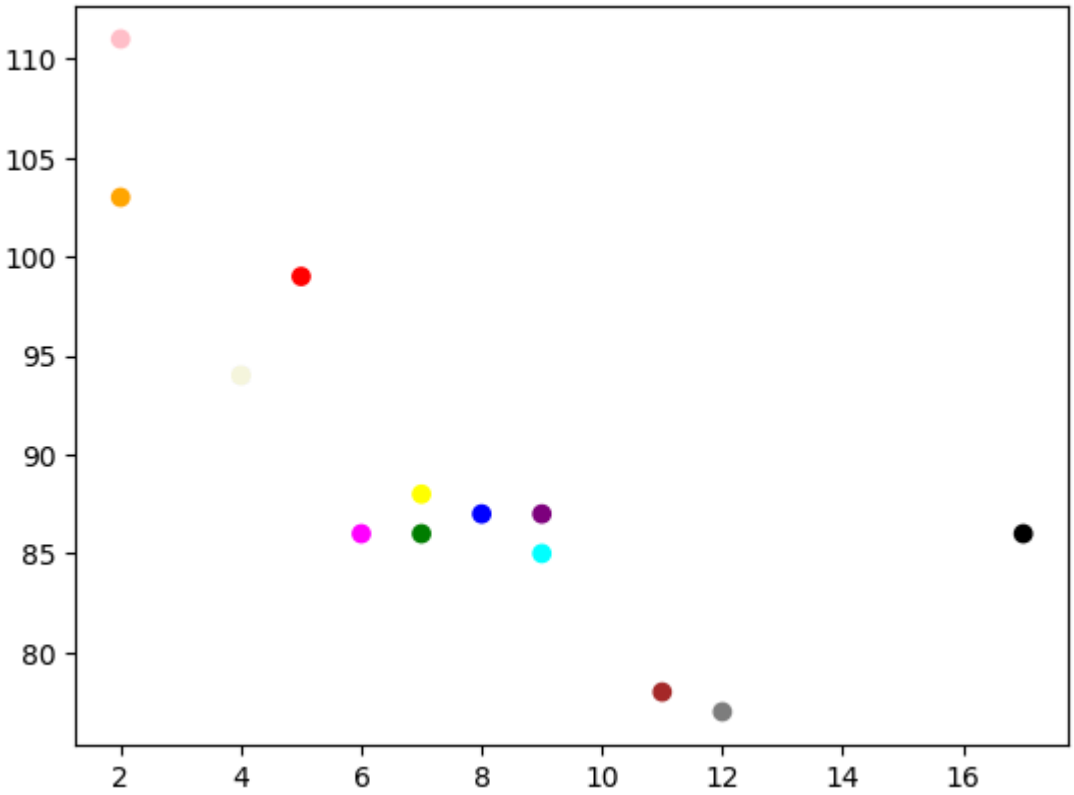

```
=== WHEN TO USE SUBPLOTS ===
✅ Comparing related datasets
✅ Showing different views of same data
✅ Before/after comparisons
✅ Different plot types for comprehensive analysis
✅ Time series with different time periods
✅ A/B testing results
✅ Multi-variable analysis

=== SUBPLOT BEST PRACTICES ===
• Use consistent scales when comparing data
• Add clear titles to each subplot
• Use plt.tight_layout() to prevent overlap
• Consider color consistency across subplots
• Don't overcrowd - limit to 2×2 or 2×3 for readability
• Use fig.suptitle() for overall figure title
```

Creating Scatter Plots

In [15]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])

plt.scatter(x, y)
plt.show()
```



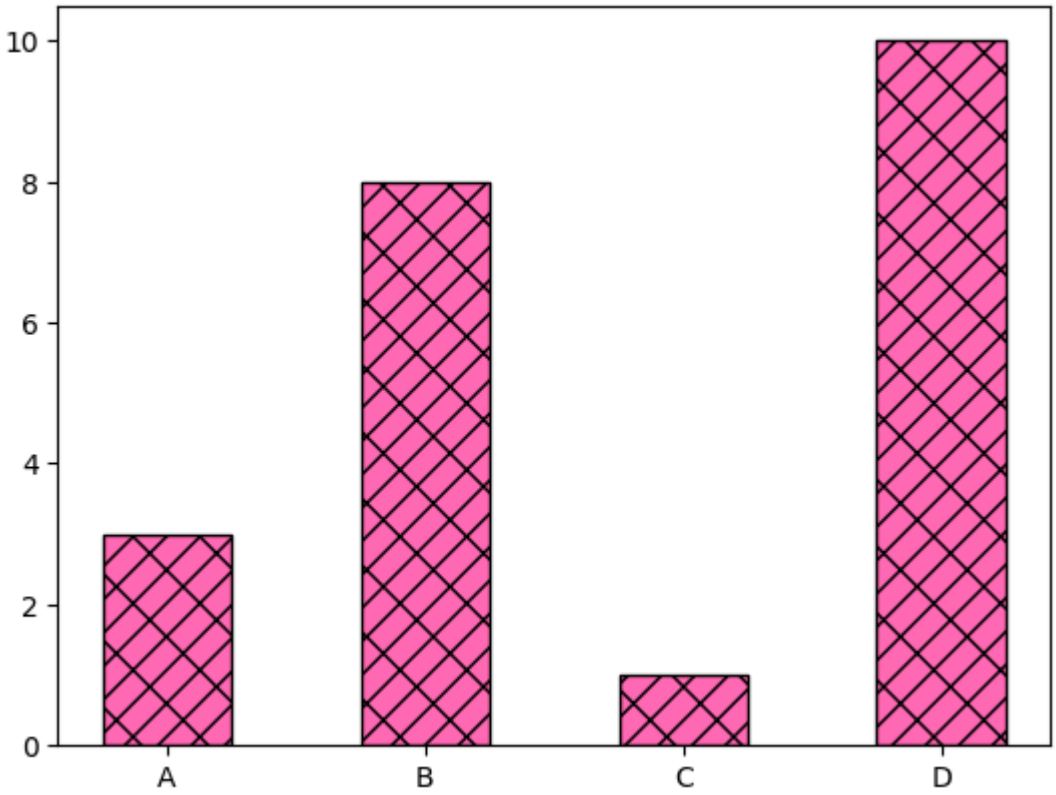
In [39]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
colors = np.array(["red","green","blue","yellow","pink","black","orange",

plt.scatter(x, y, c=colors)

plt.show()
```



Bar chart with Color Names

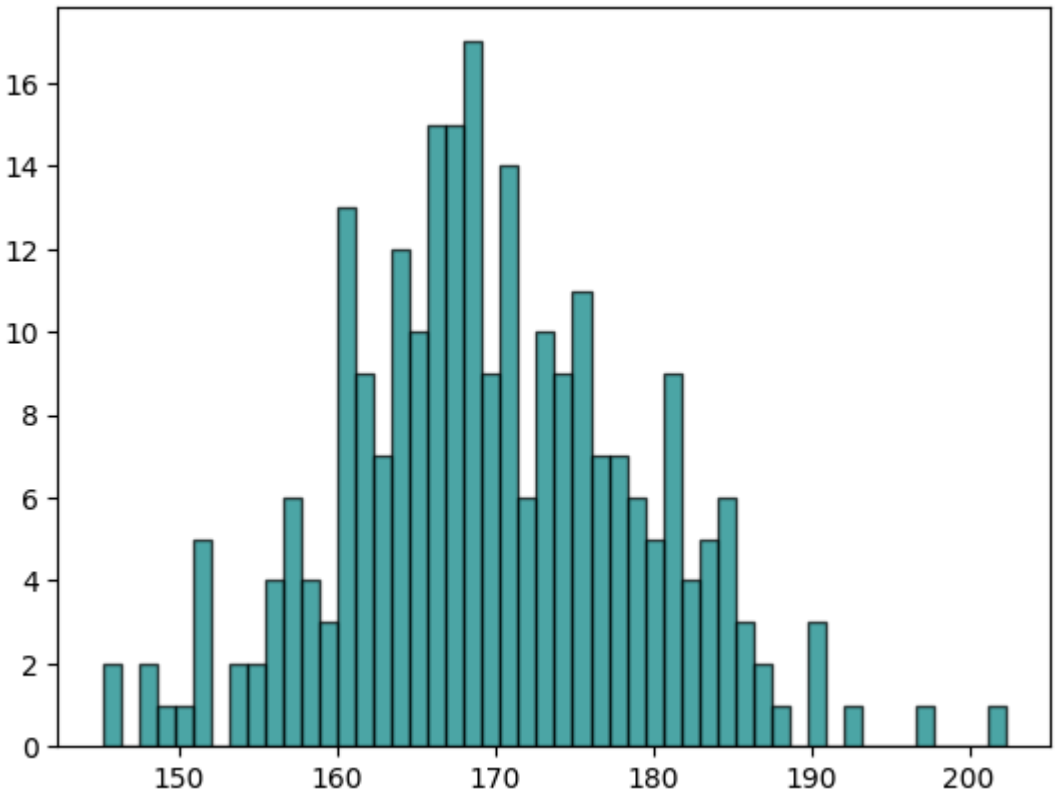
In [ ]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x, y, color = "hotpink",    width = 0.5,    edgecolor = "black",
        linewidth = 1,
        hatch = "/") # pattern inside the bars it can be /, \, |, -, +, x
plt.show()
```

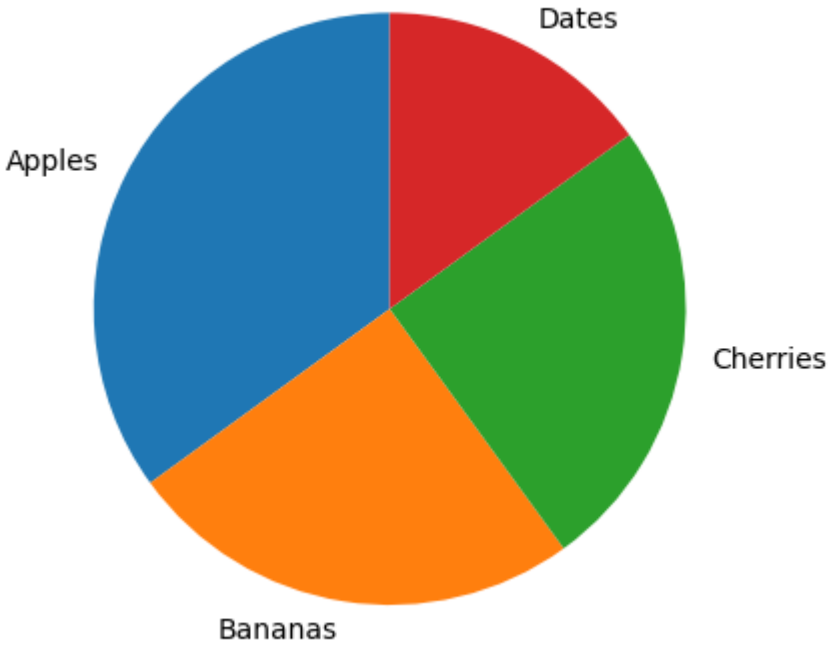
```
/var/folders/0j/xl7l0bxj4055n123kt409zjm27pcyc/T/ipykernel_28691/122975342
8.py:7: MatplotlibDeprecationWarning: hatch must consist of a string of "*
+-./OX\ox|" or None, but found the following invalid values "a". Passing i
nvalid values is deprecated since 3.4 and will become an error in 3.11.
  plt.bar(x, y, color = "hotpink",    width = 0.5,    edgecolor = "black",
```



Histogram

In [62]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.random.normal(170, 10, 250)

plt.hist(x, bins = 50, color = "teal", edgecolor = "black", linewidth = 1
plt.show()
```



Pie Charts

In [61]:
```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels = mylabels, startangle = 90) #startangle rotates the st
plt.show()
```